

CODE ANALYSIS REPORT

LMDS V5.4.001

Logistics Master Data System

รายงานวิเคราะห์ระบบแบบละเอียด
สถาปัตยกรรม คุณภาพโค้ด และข้อเสนอแนะปรับปรุง

Repository: phaopanya-scgjwd-test

Version: V5.4.001 (Schema V5.4.001)

แพลตฟอร์ม: Google Apps Script + Google Sheets

วันที่วิเคราะห์: 29 พฤษภาคม 2569

สารบัญ

1. ภาพรวมระบบ

1.1 ข้อมูลทั่วไป

1.2 สถาปัตยกรรม 6 ชั้น

1.3 โมเดลข้อมูล Trinity Framework

2. โครงสร้าง Repository

2.1 โครงสร้างไฟล์

2.2 สถิติโค้ด

3. สถาปัตยกรรมระบบเชิงลึก

3.1 Core System (Group 0)

3.2 Master Database (Group 1)

3.3 Daily Operations (Group 2)

3.4 Hybrid Alias Architecture

3.5 Match Engine (8 Rules)

4. การประเมินคุณภาพโค้ด

4.1 ผลการตรวจสอบ 15 กฎ

4.2 ปัญหาที่พบ (Critical/High/Medium)

4.3 การจัดการ Error และความปลอดภัย

5. การวิเคราะห์ประสิทธิภาพ

5.1 กลไก Cache 3 ชั้น

5.2 จุดเสี่ยงด้านประสิทธิภาพ

6. ข้อเสนอแนะปรับปรุง

6.1 ระยะสั้น (Critical Fixes)

6.2 ระยะกลาง (Refactoring)

6.3 ระยะยาว (Architecture)

7. สรุป

1. ภาพรวมระบบ

1.1 ข้อมูลทั่วไป

LMDS (Logistics Master Data System) เป็นระบบจัดการข้อมูลหลักและจับคู่ข้อมูลการจัดส่งสินค้า พัฒนบนแพลตฟอร์ม Google Apps Script ทำงานร่วมกับ Google Sheets เป็นฐานข้อมูล ระบบนี้รับข้อมูลการจัดส่งจาก SCG API ที่มีคุณภาพต่ำ (ชื่อสกปรก ที่อยู่ไม่เป็นมาตรฐาน พิกัดไม่แม่นยำ) ผ่านกระบวนการทำความสะอาด ทำ Normalize จับคู่กับข้อมูลหลัก และสร้าง Transaction สำหรับการติดตามการจัดส่งจริง

22

ไฟล์โค้ด (.gs)

~8,700

บรรทัดโค้ด

20

ชุดข้อมูล

16

Schema นิยาม

1.2 สถาปัตยกรรม 6 ชั้น

ระบบออกแบบตามสถาปัตยกรรมแบบชั้น (Layered Architecture) จำนวน 6 ชั้น แต่ละชั้นมีหน้าที่ชัดเจนและทำงานแบบ One-Way Dependency (ชั้นบนเรียกชั้นล่างเท่านั้น) การแบ่งชั้นแบบนี้ช่วยให้สามารถทดสอบและบำรุงรักษาแต่ละส่วนได้อย่างอิสระ โดยไม่กระทบต่อส่วนอื่นๆ ของระบบ

Layer A

Ingestion — ดึงข้อมูลดิบจาก SCG API / Source Sheet



Layer B

Normalization — ทำความสะอาดชื่อ ที่อยู่ พิกัด



Layer C

Master Resolution — จับคู่ Person / Place / Geo กับตารางหลัก



Layer D

Hybrid Alias — จัดการ Alias แบบ Global + Entity-Specific



Layer E

Transaction & Review — บันทึก FACT / ส่งเข้าคิวรีวิว



Layer F

Governance & Hardening — ตรวจสอบ วินิจฉัย ป้องกันระบบ

1.3 โมเดลข้อมูล Trinity Framework

หัวใจของระบบคือ **Trinity Framework** ซึ่งยึดหลักการว่าการจัดส่งทุกครั้งต้องมี 3 เสาหลัก: WHO (ผู้รับ), WHERE-Address (ที่อยู่), WHERE-Coordinate (พิกัด) การออกแบบนี้ช่วยให้สามารถแยกแยะและจัดการแต่ละมิติของข้อมูลได้อย่างอิสระ โดยมีตาราง M_DESTINATION เป็นจุดตัดที่เชื่อมทั้ง 3 เสาเข้าด้วยกัน พร้อมรหัส master_uuid ที่ใช้ในการเชื่อมโยง Alias ข้ามเอนทิตี

เสาหลัก	ตาราง	คอลัมน์สำคัญ	จำนวนคอลัมน์
WHO	M_PERSON	person_id, canonical_name, normalized_name, master_uuid	10
WHERE-Address	M_PLACE	place_id, canonical_name, province, district, master_uuid	14
WHERE-Coordinate	M_GEO_POINT	geo_id, lat, lng, radius_m, confidence	14
จุดตัด	M_DESTINATION	dest_id, person_id, place_id, geo_id, lat, lng	11

2. โครงสร้าง Repository

2.1 โครงสร้างไฟล์

Repository ถูกจัดระเบียบตามกลุ่มงาน (Group) โดยแบ่งเป็น 3 กลุ่มหลัก บวกกับโฟลเดอร์เอกสารประกอบ การตั้งชื่อไฟล์ใช้รูปแบบ `XX_Component.gs` ซึ่งตัวเลขนำหน้าบอกลำดับการโหลด (Load Order) และชื่อหลังบอกหน้าที่ของโมดูล ตามกฎที่ 14 ของโปรเจกต์

กลุ่ม	ไฟล์	หน้าที่หลัก
Group 0: Core	00_App.gs	Entry Point / Menu / Trigger
	01_Config.gs	Constants / Index / Validation
	02_Schema.gs	Schema Definition / Header Validation
	03_SetupSheets.gs	Sheet Creation / Logging
	14_Utils.gs	UUID / ID Generator / Helpers
	19_Hardening.gs	Preflight Audit / Duplicate Detection
Group 1: Master DB	05_NormalizeService.gs	Text Normalization
	06_PersonService.gs	Person CRUD / Alias
	07_PlaceService.gs	Place CRUD / Alias
	08_GeoService.gs	Geo Point Management
	09_DestinationService.gs	Destination Management
	10_MatchEngine.gs	Core Matching + M_ALIAS Single Writer
	16_GeoDictionaryBuilder.gs	Thai Geo Dictionary
Group 2: Daily Ops	04_SourceRepository.gs	Source Data Loader
	11_TransactionService.gs	FACT_DELIVERY CRUD
	12_ReviewService.gs	Review Queue Management
	13_ReportService.gs	Data Quality Reporting
	15_GoogleMapsAPI.gs	Google Maps Integration
	17_SearchService.gs	6-Tier Search / Bridge
	18_ServiceSCG.gs	SCG API Data Pipeline
Supplementary	20_ThGeoService.gs	Thai Geography Reference
	21_AliasService.gs	Hybrid Alias Architecture

2.2 สถิติโค้ด

6	7	6	3
Core Files	Master DB Files	Daily Ops Files	Supplementary Files

สัดส่วนโค้ดกระจายอยู่อย่างสมดุลระหว่าง 3 กลุ่ม โดย Group 1 (Master DB) มีโค้ดมากที่สุดประมาณ 4,100 บรรทัด เนื่องจากเป็นกลุ่มที่มี Logic การจับคู่และ Alias Management ที่ซับซ้อน Group 0 (Core) มีประมาณ 2,830 บรรทัด และ Group 2 (Daily Ops) มีประมาณ 2,470 บรรทัดตามลำดับ นอกจากนี้ยังมีไฟล์เอกสาร 17 ไฟล์ในโฟลเดอร์ docs/ ครอบคลุม Blueprint, กฎการเขียนโค้ด, แผนปรับปรุง และรายงาน Cross-Validation

3. สถาปัตยกรรมระบบเชิงลึก

3.1 Core System (Group 0)

Core System เป็นรากฐานของระบบทั้งหมด ประกอบด้วย 6 ไฟล์ที่ต้องถูกโหลดก่อนไฟล์อื่นๆ ตามลำดับ หมายเลข `01_Config.gs` เป็น Single Source of Truth สำหรับค่าคงที่ทั้งหมด รวมถึงชื่อชีต 21 ชีต, Index Constants 13 ชุด (PERSON_IDX, PLACE_IDX, FACT_IDX ฯลฯ), AI_CONFIG สำหรับควบคุมการจับคู่, และ SCG_CONFIG สำหรับการเชื่อมต่อ API

กลไก `validateConfig()`: ทำงานทุกครั้งที่เปิด Spreadsheet (ผ่าน `onOpen`) ตรวจสอบว่า `THRESHOLD_AUTO > THRESHOLD_REVIEW > THRESHOLD_IGNORE` และตรวจสอบ `SCHEMA.length` vs `IDX.keys` ตรงกันทุกตาราง หากไม่ตรงจะโยน Error ทันทีเพื่อป้องกันการดำเนินงานด้วย Config ที่ผิดพลาด

`00_App.gs` เป็น Gateway หลัก ควบคุม Custom Menu และ Pipeline Trigger ฟังก์ชันสำคัญคือ `runFullPipeline()` ที่รัน 3 ขั้นตอนต่อเนื่อง: Load Source, Normalize, Match Engine โดยมี LockService ป้องกันการรันซ้อน, Cache Invalidation ก่อนเริ่ม Pipeline, และ Diagnostic Summary หลังเสร็จสิ้น

3.2 Master Database (Group 1)

กลุ่มนี้รับผิดชอบการจัดการข้อมูลหลักทั้งหมด ตั้งแต่การทำ Normalize ชื่อและที่อยู่, การสร้าง/อัปเดต Master Record, ไปจนถึงการจับคู่ข้อมูลกับ Master ที่มีอยู่ โมดูลสำคัญที่สุดคือ `10_MatchEngine.gs` ซึ่งเป็นหัวใจของ Pipeline และเป็น Single Writer เดียวสำหรับตาราง M_ALIAS ตามรูปแบบ Single Writer Pattern

บริการ Normalize (`05_NormalizeService.gs`)

ทำหน้าที่ทำความสะอาดข้อความทั้งหมดก่อนเข้าสู่กระบวนการจับคู่ รวมถึงการตัดช่องว่าง, แปลงเป็นตัวพิมพ์เล็ก, ลบคำนำหน้า/คำลงท้ายที่ไม่จำเป็น, และ Normalization ภาษาไทยเฉพาะ เช่น การจัดการสระและ

วรรณยุกต์ ข้อมูลที่ผ่าน Normalize แล้วจะถูกเก็บในคอลัมน์ normalized_name แยกจาก canonical_name (ชื่อเดิม) เพื่อให้สามารถค้นหาเปรียบเทียบได้โดยไม่สูญเสียข้อมูลต้นฉบับ

บริการ Person & Place (06, 07)

รับผิดชอบการสร้าง, ค้นหา, อัปเดต Master Record ของบุคคลและสถานที่ ทั้งสองโมดูลมีกลไก Cache ภายในเพื่อลดการอ่านซ้ำ และใช้ `master_uuid` เป็นตัวเชื่อมโยงกับระบบ Aliasสากล ฟังก์ชันสำคัญได้แก่ `resolvePerson()` และ `resolvePlace()` ที่ทำการค้นหา + Fuzzy Match กับข้อมูลที่มีอยู่ ก่อนจะตัดสินใจว่าจะใช้ Record เดิมหรือสร้างใหม่

3.3 Daily Operations (Group 2)

กลุ่มนี้รับผิดชอบงานประจำวันที่ต้องทำซ้ำทุกวัน โดยเฉพาะการดึงข้อมูลจาก SCG API และการจับคู่พิกัด `18_ServiceSCG.gs` เป็น Commander ของกลุ่ม ทำหน้าที่ดึงข้อมูลจาก SCG API ด้วย Cookie Authentication, แปลง JSON เป็น Flat Rows, คำนวณสรุปตามร้านค้า, เรียก Module 17 จับคู่พิกัด, และสร้างสรุปเจ้าของสินค้า/Shipment ใน V5.4.002 ได้ลบการเรียก `populateAliasFromSCGRawData_()` ออกจาก Pipeline ของ Group 2 เพื่อปฏิบัติตาม Single Writer Pattern

3.4 Hybrid Alias Architecture

นวัตกรรมสำคัญของ V5.4 คือระบบ Hybrid Alias ที่ผสมผสานข้อดีของ Entity-Specific Alias (V5.2) เข้ากับ Global Alias Ledger ตารางกลาง `M_ALIAS` ทำหน้าที่เป็น Single Source of Truth สำหรับการค้นหา Alias ข้ามเอนทิตี โดยมี `master_uuid` เป็นกุญแจเชื่อมโยง สถาปัตยกรรมนี้แก้ปัญหาลำบาก 3 ประการ: ประการแรกคือ Alias ที่อยู่คนละ Silo ไม่สามารถค้นหาข้ามเอนทิตีได้ใน V5.2, ประการที่สองคือ $O(N)$ Scan ใน V4.0 ที่ช้ามากเมื่อข้อมูลเพิ่มขึ้น, และประการที่สามคือไม่มี Fast Track สำหรับการค้นหา ShipToName ที่ใช้บ่อยในงานประจำวัน

Single Writer Pattern: ฟังก์ชัน `autoEnrichAliasesFromFactBatch_()` ใน `10_MatchEngine.gs` เป็นจุดเขียนเดียวสำหรับ `M_ALIAS` ใน Auto Pipeline ฟังก์ชันอื่นๆ สามารถอ่าน `M_ALIAS` ได้ แต่ห้ามเขียนโดยตรงเด็ดขาดในระหว่าง Pipeline ทำงาน เพื่อป้องกัน Race Condition และ Data Inconsistency การบังคับใช้ Single Writer นี้ทำให้สามารถรับประกันความสม่ำเสมอของข้อมูล Alias ได้ แม้จะมีหลายกระบวนการทำงานพร้อมกัน

3.5 Match Engine (8 Rules)

Match Engine เป็นหัวใจหลักของ Pipeline ใช้กฎลำดับชั้น (Priority Rules) จำนวน 8 ข้อในการตัดสินใจ โดยประเมินจากกรณีที่แย่ที่สุดไปหาดีที่สุด กฎแต่ละข้อมี Priority Level กำหนดความเร่งด่วน และ Confidence Score บอกระดับความมั่นใจในการตัดสินใจ การออกแบบแบบลำดับชั้นช่วยให้กรณีที่มียุทธวิธีร้ายแรง (เช่น พิกัดไม่ถูกต้อง) ถูกจัดคิวไว้ก่อนกรณีที่มีความกำกวมน้อยกว่า

ลำดับ	กฎ	Action	Priority	เงื่อนไข
1	INVALID_LATLNG	REVIEW	1	ไม่มีพิกัดใน Source (0,0 หรือว่าง)
2	LOW_QUALITY_DATA	REVIEW	2	ชื่อสั้นเกินไปหรือคุณภาพต่ำ
3	GEO_PROVINCE_CONFLICT	REVIEW	2	จังหวัดจากพิกัดไม่ตรงกับจังหวัดในที่อยู่
3.5	NEARBY_PENDING	REVIEW	1	พิกัดใกล้เคียงกัน ต้องมีคนตัดสินใจ
4	FULL_MATCH	AUTO_MATCH	0	พบครบทั้ง Person + Place + Geo
5	GEO_ANCHOR	AUTO_MATCH	0	พบ Geo + อย่างน้อย 1 อย่าง (Person หรือ Place)
6	FUZZY_MATCH	REVIEW	2	มีความกำกวม ต้องมีคนตรวจสอบ
7	ALL_NEW_WITH_GEO	CREATE_NEW	0	ข้อมูลใหม่ทั้งหมดแต่มีพิกัดถูกต้อง
8	DEFAULT	REVIEW	3	กรณีอื่นๆ ที่ไม่เข้าเงื่อนไขใดๆ

กฎที่ 4 (FULL_MATCH) ใช้ Weight รวม: Geo 50% + Person 30% + Place 20% สะท้อนว่าพิกัดมีน้ำหนักสูงสุดเพราะเป็นตัวบ่งชี้ที่เชื่อถือได้มากที่สุดในบริบทขนส่ง ส่วนกฎที่ 5 (GEO_ANCHOR) ปรับน้ำหนักเป็น Geo 60% + Person/Place 40% เพราะเมื่อมีเพียงบางส่วนที่จับคู่ได้ พิกัดจะมีบทบาทสำคัญยิ่งขึ้นในการยืนยันว่าเป็นปลายทางเดียวกัน

4. การประเมินคุณภาพโค้ด

4.1 ผลการตรวจสอบ 15 กฎ

โปรเจกต์กำหนดกฎการเขียนโค้ด 15 ข้อที่ทุกไฟล์ต้องปฏิบัติตาม การวิเคราะห์พบว่ามี 6 ข้อผ่าน, 4 ข้อผ่านบางส่วน, และ 3 ข้อไม่ผ่าน สะท้อนว่าทีมพัฒนาให้ความสำคัญกับคุณภาพโค้ดในระดับหนึ่ง แต่ยังมีจุดที่ต้องปรับปรุง โดยเฉพาะในด้าน Single Responsibility, No Cross-File Globals และ Error Handling

กฎ	ชื่อ	สถานะ	หมายเหตุ
#1	Clean Code	PARTIAL	ฟังก์ชันบางตัวยาวเกิน 50 บรรทัด
#2	Single Responsibility	FAIL	autoEnrichAliasesFromFactBatch_ ยาว 266 บรรทัด ทำหลายหน้าที่
#3	No Hardcode Index	PASS	ใช้ XXX_IDX constants ทั้งหมด
#4	Batch Operations Only	PARTIAL	createGlobalAlias() ยังใช้ appendRow()
#5	Checkpoint & Resume	PASS	มี Time Guard + Auto-Resume Trigger
#7	No Fake Function Calls	PASS	แก้ไข autoInstallSmartNav_() แล้ว
#8	Namespace Pattern	PARTIAL	บางฟังก์ชันยังอยู่ใน Global Scope
#9	No Global State	FAIL	_GLOBAL_* Cache อยู่ใน Global Scope
#10	Lock Library Version	PASS	ไม่ใช้ HEAD reference
#12	Error Handling	FAIL	21/26 Entry Points ขาด try-catch
#13	Logging with Context	PARTIAL	บางจุดขาด Stack Trace
#14	Structured File Names	PASS	ใช้ XX_Component.gs ทั้งหมด
#15	Full Files Only	PASS	ไม่มี "..." หรือ "// old code"

4.2 ปัญหาที่พบ

Critical Issues

C1: 21/26 Entry Points ขาด try-catch

ฟังก์ชันที่ถูกเรียกจาก Menu/Trigger จำนวน 21 จาก 26 ฟังก์ชัน ไม่มีการห่อด้วย try-catch ทำให้เมื่อเกิด Error ระบบจะหยุดทำงานทันทีโดยไม่บันทึก Log และไม่แจ้งเตือนผู้ใช้อย่างเหมาะสม สิ่งนี้ขัดต่อกฎที่ 12 (Error Handling per Entry Point) และอาจทำให้ข้อมูลอยู่ในสถานะกกลาง (Inconsistent State) หาก Error เกิดระหว่าง Batch Write

C2: autoEnrichAliasesFromFactBatch_ ยาว 266 บรรทัด

ฟังก์ชันนี้ทำหน้าที่หลายอย่าง: โหลดข้อมูลอ้างอิง, ตรวจสอบ Dedup, สร้าง Alias Rows สำหรับ 3 ชีต, และ

Batch Write ทั้งหมด ขัดต่อกฎที่ 2 (Single Responsibility) และกฎที่ 1 (Clean Code ฟังก์ชัน 30-50 บรรทัด) ฟังก์ชันที่ยาวเกินไปยากต่อการทดสอบ บำรุงรักษา และทำ Code Review

High Priority Issues

H1: createGlobalAlias() ใช้ appendRow() แทน Batch Write

ฟังก์ชัน createGlobalAlias() ใช้ sheet.appendRow() ซึ่งเป็นการเขียนที่ละแถว ขัดต่อกฎที่ 4 (Batch Operations Only) เมื่อ Migration ต้องสร้าง Alias หลายร้อยรายการ appendRow() จะช้ามากเมื่อเทียบกับ setValues() แบบ Batch อย่างไรก็ตาม ฟังก์ชันนี้ถูกจำกัดการใช้งานเฉพาะ Migration/Admin เท่านั้นใน V5.4 จึงไม่ส่งผลกระทบต่อ Pipeline หลัก

H2: 5 ฟังก์ชันขาด Time Guard

ฟังก์ชันที่ประมวลผลนาน เช่น buildGeoDictionary(), populateGeoMetadata() ไม่มีการตรวจสอบเวลาแบบ hasTimePassed() ทำให้เสี่ยงต่อ GAS Timeout (6 นาที) โดยไม่มีกลไก Resume ในขณะที่ runMatchEngine() มี Time Guard + Auto-Resume ครบถ้วน ฟังก์ชันที่ทำงานนานอื่นๆ ยังขาดการป้องกันนี้

Medium Priority Issues

M1: _GLOBAL_* Cache อยู่ใน Global Scope

ตัวแปร _GLOBAL_GEO_DICT_CACHE และ _GLOBAL_GEO_POINTS_CACHE ประกาศเป็น Global Variable ใน 01_Config.gs ขัดต่อกฎที่ 9 (No Cross-File Globals) ควรย้ายเข้าใน Module ของตัวเองหรือใช้ Closure Pattern เพื่อลด Coupling ระหว่างไฟล์

M2: MAPS_CACHE_IDX ขาดนิยาม

ใน 15_GoogleMapsAPI.gs มีการอ้างอิง MAPS_CACHE_IDX แต่ไม่มีนิยามใน 01_Config.gs อาจเป็น Bug ที่ยังไม่แสดงอาการเพราะฟังก์ชันที่ใช้ยังไม่ถูกเรียก หรือมี Fallback อื่นอยู่ แต่ควรตรวจสอบและเพิ่มนิยามให้ครบถ้วนตาม Schema

4.3 การจัดการ Error และความปลอดภัย

การจัดการ Error

ระบบมีกลไก Logging ผ่าน SYS_LOG ที่บันทึกทุก Error พร้อม Module, Level, Message และ Details อย่างไรก็ดี ยังมีจุดที่ควรปรับปรุง: ฟังก์ชัน safeRun() ใน 00_App.gs เป็น Global Error Handler ที่ดี แต่ยังไม่ถูกนำไปใช้ครอบคลุมทุก Entry Point นอกจากนี้ Error ใน autoEnrichAliasesFromFactBatch_ ถูกห่อด้วย try-catch แล้วใน V5.4.001 แต่ Error ในฟังก์ชันอื่นๆ ของ MatchEngine ยังไม่มีการจัดการที่ครบถ้วน

ความปลอดภัย

ระบบมีมาตรการรักษาความปลอดภัยหลายชั้น: API Key ถูกเก็บใน PropertiesService (ไม่ Hardcode ในโค้ด), Cookie สำหรับ SCG API ถูกเก็บในชิต Input แยกจากโค้ด, และมี validateConfig() ตรวจสอบ API Key Format อย่างไรก็ตาม มีจุดเสี่ยงที่ควรพิจารณา: GitHub Token ถูก Hardcode ในคำสั่ง Clone (ควรใช้ Environment Variable หรือ SSH Key แทน), และ Cookie Authentication กับ SCG API ไม่มีการเข้ารหัสหรือหมดอายุอัตโนมัติ

5. การวิเคราะห์ประสิทธิภาพ

5.1 กลไก Cache 3 ชั้น

ระบบใช้กลไก Cache 3 ชั้นเพื่อลดการอ่าน Google Sheets ที่ซ้ำ: ชั้นที่ 1 เป็น RAM Cache ใช้ Global Variable ใน Script เก็บข้อมูลที่ใช้อยู่ในหน่วยความจำ ชั้นที่ 2 เป็น CacheService ของ GAS เก็บข้อมูลแบบ Serialized JSON ด้วย TTL 6 ชั่วโมง และมีการ Chunk สำหรับข้อมูลที่ใหญ่เกิน 100KB ชั้นที่ 3 เป็น Sheet (Persistent) อ่านจาก Google Sheets โดยตรงเมื่อ Cache ไม่มีข้อมูล การออกแบบ 3 ชั้นนี้ช่วยลดเวลาตอบสนองจากหลายวินาที (อ่าน Sheet) เหลือมิลลิวินาที (RAM Cache) สำหรับข้อมูลที่ใช้ซ้ำ

ชั้น	ตัวเก็บ	ความเร็ว	ข้อจำกัด	ใช้สำหรับ
1 - RAM	Global Variables	~0ms	รีเซ็ตเมื่อ Script รีสตาร์ท	Geo Dict, Geo Points
2 - CacheService	GAS Cache	~5-50ms	TTL 6hr, ขนาดสูงสุด 100KB/key	M_ALIAS, Person, Place
3 - Sheet	Google Sheets	~100-500ms	ช้าที่สุด แต่ถาวร	ข้อมูลทั้งหมด

5.2 จุดเสี่ยงด้านประสิทธิภาพ

P1: O(N) Scan ใน resolveMasterUidViaGlobalAlias()

ฟังก์ชัน resolveMasterUidViaGlobalAlias() ใน 21_AliasService.gs ทำการวนลูปผ่าน aliasesMap ทั้งหมดเพื่อค้นหา Best Match ซึ่งมี Complexity เป็น O(N) เมื่อจำนวน Alias เพิ่มขึ้นเป็นหลักพัน ฟังก์ชันนี้จะช้าลงอย่างมาก ควรปรับเป็น O(1) Lookup โดยใช้ Reverse Index (ซึ่ง loadGlobalAliasReverseIndex_() มีอยู่แล้ว) แทนการวนลูปทั้ง Map

P2: Stats Update แบบ Row-by-Row

ฟังก์ชัน updatePersonStats(), updatePlaceStats(), updateGeoStats() และ updateDestinationStats() อัปเดตสถิติการใช้งาน (usage_count, last_seen) โดยใช้ getValue()/setValue() ทีละแถว แม้ V5.4.002 จะได้ แก้ updateDestinationStats() เป็น Batch แล้ว แต่ฟังก์ชัน Stats อื่นๆ ยังใช้วิธีเดิม เมื่อจำนวนการจับคู่ต่อ รอบมีมาก (50-200 รายการ) การอัปเดตทีละแถวจะสะสมเวลาทำให้เกิน GAS Timeout

P3: การโหลดข้อมูลทั้งชีตใน autoEnrichAliasesFromFactBatch_

ฟังก์ชัน autoEnrichAliasesFromFactBatch_() โหลด allPersons, allPlaces, allAliases, allPlaceAliases ทุก ครั้งที่ถูกเรียก (ทุก 20 แถวตาม BATCH_SIZE) แม้จะมี Cache ช่วย แต่การ Parse JSON จาก CacheService ทุก ครั้งก็ยังใช้เวลา ควรพิจารณาโหลดครั้งเดียวแล้วส่งผ่านเป็น Parameter หรือใช้ RAM Cache ที่อยู่ใน Script Run ปัจจุบัน

6. ข้อเสนอแนะปรับปรุง

6.1 ระยะสั้น - Critical Fixes (1-2 สัปดาห์)

รหัส	ปัญหา	แนวทางแก้ไข	ความสำคัญ
S1	Entry Points ขาด try-catch	ห่อทุก Entry Point ด้วย safeRun() wrapper หรือเพิ่ม try-catch โดยตรง ดำเนินการทั้ง 21 ฟังก์ชัน	CRITICAL
S2	ฟังก์ชันขาด Time Guard	เพิ่ม hasTimePassed_() + installAutoResume_() ใน buildGeoDictionary(), populateGeoMetadata() และอีก 3 ฟังก์ชัน	CRITICAL
S3	MAPS_CACHE_IDX ขาดนิยาม	เพิ่ม MAPS_CACHE_IDX ใน 01_Config.gs ให้ตรงกับ SCHEMA['MAPS_CACHE'] (10 คอลัมน์)	HIGH
S4	Stats Update row-by-row	แปลง updatePersonStats/updatePlaceStats/updateGeoStats เป็น Batch Write แบบ updateDestinationStats ที่แก้แล้ว	HIGH

6.2 ระยะกลาง - Refactoring (3-4 สัปดาห์)

รหัส	ปัญหา	แนวทางแก้ไข	ความสำคัญ
M1	autoEnrichAliasesFromFactBatch_ ยาว 266 บรรทัด	แตกเป็น 4 ฟังก์ชันย่อย: buildPersonAliases_(), buildPlaceAliases_(), deduplicateAliases_(), writeAliasBatches_()	HIGH
M2	resolveMasterUuidViaGlobalAlias เป็น O(N)	แทนที่ด้วย loadGlobalAliasReverseIndex_() สำหรับ O(1) Lookup โดยตรง	MEDIUM
M3	createGlobalAlias() ใช้ appendRow()	สร้าง Batch Variant: createGlobalAliasBatch_() ที่ใช้ setValues() และเก็บสะสมแถวก่อนเขียนทีเดียว	MEDIUM
M4	Global Variables _GLOBAL_*	ย้ายเข้า Closure Pattern หรือสร้าง CacheManager Module แยกต่างหาก	MEDIUM
M5	Cache Read Pattern ซ้ำ 8 จุด	สร้าง genericCachedLoader_(cacheKey, loaderFn) ที่รวม RAM Cache + CacheService + Sheet Read เป็นฟังก์ชันเดียว	MEDIUM

6.3 ระยะยาว - Architecture (5-8 สัปดาห์)

รหัส	ปัญหา	แนวทางแก้ไข	ความสำคัญ
L1	Google Sheets เป็น Database มีข้อจำกัด	พิจารณาย้าย Master Data ไป Cloud SQL / Firestore เพื่อรองรับ Query ที่ซับซ้อนขึ้นและ Concurrent Access ที่ดีกว่า โดยยังคงใช้ Sheets เป็น UI ผ่าน Connected Sheets	MEDIUM
L2	ไม่มี Automated Testing	สร้าง Test Suite ด้วย GAS Clasp + Jest สำหรับ Unit Test ของ Match Engine, Normalization, Alias Resolution และ Integration Test สำหรับ Pipeline ทั้งหมด	MEDIUM
L3	Cookie Authentication ไม่ปลอดภัย	พัฒนา OAuth2 Flow หรือใช้ API Key Authentication แทน Cookie ที่หมดอายุและต้องอัปเดตเอง	HIGH
L4	ไม่มี CI/CD Pipeline	ตั้งค่า GitHub Actions สำหรับ Lint, Test, Deploy อัตโนมัติผ่าน Clasp Push เพื่อลด Human Error ในการ Deploy	MEDIUM

7. สรุป

LMDS V5.4.001 เป็นระบบที่ออกแบบมาอย่างดีสำหรับการจัดการข้อมูลหลักด้านโลจิสติกส์บนแพลตฟอร์ม Google Apps Script สถาปัตยกรรม 6 ชั้นมีการแบ่งแยกหน้าที่ชัดเจน, Trinity Framework แก้ปัญหาข้อมูลที่ซ้ำซ้อนและไม่สอดคล้อง, และ Hybrid Alias Architecture เป็นนวัตกรรมที่แก้ปัญหาค้นหาข้ามเอนทิตีได้อย่างมีประสิทธิภาพ กฎการเขียนโค้ด 15 ข้อแสดงถึงความใส่ใจในคุณภาพ และกระบวนการ Cross-Validation ระหว่าง 5 ระบบ AI แสดงถึงความรอบคอบในการตรวจสอบ

อย่างไรก็ตาม ยังมีปัญหาสำคัญที่ต้องแก้ไข: การขาด try-catch ที่ Entry Points ส่วนใหญ่เป็นความเสี่ยงสูงที่อาจทำให้ระบบหยุดทำงานโดยไม่มีการแจ้งเตือน ฟังก์ชันหลักที่ยาวเกินไป (266 บรรทัด) ทำให้ยากต่อการบำรุงรักษา และจุดเสี่ยงด้านประสิทธิภาพหลายจุดอาจกลายเป็นปัญหาเมื่อข้อมูลเพิ่มขึ้น ข้อเสนอแนะในรายงานนี้จัดลำดับความสำคัญเป็น 3 ระยะ เพื่อให้ทีมพัฒนาสามารถดำเนินการได้อย่างเป็นขั้นตอน โดยเริ่มจาก Critical Fixes ที่ส่งผลกระทบต่อความเสถียรของระบบก่อน

คะแนนรวม: โค้ดมีคุณภาพดีในระดับ 7/10 สถาปัตยกรรมแข็งแกร่ง กฎการเขียนโค้ดชัดเจน และมีกระบวนการตรวจสอบที่ดี จุดที่ต้องปรับปรุงหลักคือ Error Handling, การ Refactor ฟังก์ชันที่ยาวเกินไป และการปรับปรุงประสิทธิภาพในจุดที่ใช้ O(N) Scan หากสามารถแก้ไข Critical Issues ทั้งหมดได้ คะแนนจะเพิ่มขึ้นเป็น 8.5/10 ได้